FINSIGHT RAG

Enterprise Financial Document Intelligence and Question Answering

PROJECT REPORT

Project Domain	Enterprise Knowledge Management / Generative AI
Core Approach	Retrieval-Augmented Generation (RAG)
Documents	Microsoft Annual Reports FY2023, FY2024 and FY2025
Application	Streamlit Web Application
Repository / Deployment	GitHub + Streamlit Community Cloud

1. Executive Summary

FinSight RAG is an enterprise financial document question-answering system designed to retrieve reliable evidence from Microsoft annual reports and generate concise answers grounded in those documents. The project addresses a practical limitation of general-purpose language models in financial analysis: a fluent answer is not sufficient when the underlying number must be traceable to a source document.

The system processes three Microsoft annual reports covering fiscal years 2023, 2024 and 2025. The reports are extracted into text, divided into metadata-aware chunks, encoded with the Sentence Transformer model all-MiniLM-L6-v2, indexed using FAISS, and retrieved through semantic similarity. A verification layer applies scope and fiscal-year checks before retrieved evidence is passed to google/flan-t5-base for grounded generation. The Streamlit interface displays the answer together with source document and fiscal-year information.

The final application was tested with direct financial questions, year-filtered questions and out-of-scope questions. The project therefore demonstrates the complete path from unstructured documents to a deployed RAG assistant, while explicitly handling cases where the available evidence is insufficient.

2. Problem Statement

Financial information is distributed across lengthy annual reports containing narrative discussion, tables, segment information and repeated figures. Manually locating a specific number or comparison is time-consuming. At the same time, a general language model may produce a plausible response that is not supported by the company documents or may confuse total company figures with segment-level metrics.

The problem addressed by FinSight RAG is therefore to build a focused document intelligence system that can retrieve relevant evidence from annual reports, answer financial questions using that evidence, identify the fiscal year being discussed, attribute the answer to its source document and refuse to provide unsupported information.

3. Objectives

- Build a complete Retrieval-Augmented Generation pipeline for enterprise financial documents.
- Extract and preprocess Microsoft annual reports into searchable text chunks.
- Preserve document metadata, especially fiscal year, during chunk creation.
- Generate dense semantic embeddings and perform similarity search using FAISS.
- Use grounded language generation so that answers are based on retrieved evidence rather than general model knowledge.
- Provide source attribution and fiscal-year filtering in a user-facing Streamlit application.
- Return an insufficient-evidence response when a question is outside the document scope or unsupported by the retrieved evidence.

4. Scope and Document Collection

The project is intentionally limited to Microsoft annual reports for fiscal years 2023, 2024 and 2025. This restricted corpus makes the system suitable for demonstrating document retrieval, metadata-aware search, grounded generation and evidence verification in a controlled enterprise setting.

Document	Fiscal year	Role in system
2023_Annual_Report.docx	FY2023	Source document for retrieval and evidence
2024_Annual_Report.docx	FY2024	Source document for retrieval and evidence
2025_AnnualReport.docx	FY2025	Source document for retrieval and evidence

The documents are public corporate reports and are used as the project knowledge base. The repository stores processed artifacts required for the deployed application rather than pretending that the system supports unrestricted web-wide financial research.

5. System Architecture

The architecture is divided into two connected stages. The first is an offline document-processing stage that creates the searchable knowledge base. The second is an online query stage that retrieves evidence and generates the final response.

1. Reports	2. Extraction	3. Chunking	4. Embeddings	5. FAISS	6. Retrieval	7. Verification	8. FLAN-T5	9. Output
Microsoft annual reports FY2023-FY2025	DOCX -> clean text	500 words 100-word overlap + metadata	all-MiniLM-L6-v2 384 dimensions	IndexFlatIP normalized vectors	semantic Top-K search	scope + evidence + fiscal year	google/flan-t5-base grounded generation	answer + source + fiscal year

Online query flow: User Question -> Query Embedding -> Retrieval -> Evidence Verification -> FLAN-T5 -> Grounded Answer + Sources

6. Methodology

6.1 Document Extraction

The annual reports are stored as DOCX files. The extraction stage reads the document content and converts it into clean text suitable for chunking. The three source documents yielded a combined searchable corpus that was then represented as structured document records.

6.2 Metadata-Aware Chunking

The extracted text is divided into chunks of 500 words with an overlap of 100 words. Each chunk is stored as a dictionary containing text, document name and fiscal year. This metadata is important because two documents may contain similar terminology while referring to different reporting periods.

6.3 Semantic Embeddings

The project uses Sentence Transformers with all-MiniLM-L6-v2. Each text chunk is transformed into a 384-dimensional dense vector. Vectors are normalized before indexing, enabling inner-product search to behave as cosine-similarity retrieval for the normalized embeddings.

6.4 FAISS Vector Search

FAISS is used as the vector index through IndexFlatIP. The index stores the normalized embeddings and returns the most similar chunks for a user query. The application retrieves a larger candidate set before applying fiscal-year constraints so that metadata filtering does not unnecessarily reduce recall.

6.5 Evidence Verification and Hallucination Handling

FinSight uses explicit scope rules and evidence checks before displaying an answer. Questions about external companies are rejected because the knowledge base is limited to Microsoft. Questions whose requested year is not represented in the corpus are also rejected. For grounded generation, the prompt instructs the language model to use only retrieved evidence, avoid outside knowledge, avoid invented numbers and distinguish total company revenue from segment or product revenue.

6.6 Grounded Generation

The generation layer uses google/flan-t5-base. The prompt contains the user question and retrieved evidence, including document name and fiscal year. The model is instructed to answer only from the supplied evidence. When no sufficient evidence is available, the application returns a fixed insufficient-evidence response instead of forcing a speculative answer.

6.7 User Interface and Filtering

The Streamlit interface presents the system as a lightweight enterprise document assistant. A sidebar identifies the report collection and provides a fiscal-year filter. The main interface accepts the user question, shows the answer status, and exposes the supporting source documents and evidence snippets.

7. Technology Stack

Component	Technology	Purpose
Programming	Python	Core implementation
Document processing	python-docx / text processing	DOCX extraction and cleaning
Embedding model	Sentence Transformers: all-MiniLM-L6-v2	384-D semantic embeddings
Vector search	FAISS IndexFlatIP	Fast similarity retrieval
Generation	Hugging Face google/flan-t5-base	Grounded answer generation
Web application	Streamlit	Interactive user interface
Version control	GitHub	Repository and project documentation
Deployment	Streamlit Community Cloud	Public application deployment

8. Implementation Summary

Metric	Implemented value
Number of annual reports	3
Fiscal years covered	2023, 2024, 2025
Final document chunks	246
Chunk size	500 words
Chunk overlap	100 words
Embedding model	all-MiniLM-L6-v2
Embedding dimension	384
FAISS index	IndexFlatIP
Generator	google/flan-t5-base
Application	Streamlit

9. Testing and Validation

Testing focused on four practical behaviours: correct retrieval for supported Microsoft financial questions, fiscal-year filtering, source attribution and refusal to answer unsupported questions. The validation set was designed around representative questions rather than a synthetic benchmark.

Test case	Expected behaviour	Validation result
Microsoft revenue in FY2025	Return the supported revenue figure with source	Passed

Test case	Expected behaviour	Validation result
	evidence	
Microsoft revenue in FY2024 with 2024 filter	Restrict evidence to FY2024 and answer from the selected year	Passed
Microsoft revenue in FY2023	Use FY2023 evidence and avoid segment-level confusion	Passed
Tesla revenue in FY2024	Reject external-company question	Passed
Apple CEO / unrelated general fact	Reject question outside Microsoft corpus	Passed
Unsupported fiscal year	Return insufficient-evidence message	Passed

The final deployed application was checked using these behaviours. The project does not claim a universal accuracy score because the current validation set is a small, task-specific set of questions. For a production release, a larger labeled evaluation set with retrieval relevance and grounded answer metrics would be appropriate.

10. Results

The completed prototype demonstrates an end-to-end RAG workflow rather than a simple chatbot. The application can retrieve evidence from the indexed annual reports, provide fiscal-year-aware answers, show source attribution and decline unsupported questions. This is the central project result: financial answers are tied to a defined document collection and visible evidence instead of relying solely on model memory.

A representative FY2025 revenue query returns the Microsoft figure supported by the FY2025 annual report, while an external-company query such as a Tesla revenue question is rejected as outside the available evidence. The Streamlit interface also exposes the selected fiscal year and supporting documents, which improves traceability for the user.

11. Limitations

- The knowledge base currently contains only three Microsoft annual reports, so the system is not a general financial research engine.
- The current retrieval index uses dense semantic search without a hybrid keyword/semantic retrieval layer.
- The generation model is a compact FLAN-T5 model and may be less capable than larger instruction-tuned models for complex multi-step reasoning.
- Evaluation is task-specific and should be expanded before making claims about production-level reliability.
- The system is primarily designed for English annual-report content and does not implement a multilingual workflow.
- The current application is a prototype deployment and does not include enterprise authentication, audit logging or role-based access control.

12. Future Scope

- Add hybrid retrieval combining dense embeddings with lexical search.
- Introduce a dedicated reranker to improve top-K evidence quality.
- Expand the document collection to additional public financial filings and company reports.
- Add a structured evaluation framework covering retrieval relevance, groundedness and answer correctness.
- Add confidence scoring and clearer evidence-quality indicators.
- Support multi-document comparison and trend analysis across reporting periods.

- Add authentication, logging and access controls for enterprise deployment.
- Introduce document re-indexing workflows so new annual reports can be added without rebuilding the project manually.

13. Deployment

The application is deployed using Streamlit Community Cloud and the source code is maintained in GitHub. The repository contains the Streamlit application, notebook, serialized document chunks, embeddings, FAISS index, requirements file, README and project assets.

GitHub Repository: https://github.com/vedika-ugale20/FinSight-RAG

Live Application: https://vedika-ugale20-finsight-rag-app-4paiye.streamlit.app

14. Repository Structure

```
FinSight-RAG/
|-- app.py
|-- FinSight_RAG_Capstone.ipynb
|-- document_chunks.pkl
|-- embeddings.pkl
|-- financial_index.faiss
|-- requirements.txt
|-- README.md
|-- .gitignore
|-- FinSight_RAG_Project_Report.pdf
|-- FinSight_RAG_Architecture.png
`-- screenshots/
```

15. Conclusion

FinSight RAG demonstrates how Retrieval-Augmented Generation can be applied to enterprise financial documents in a controlled and explainable workflow. The system combines document extraction, metadata-aware chunking, semantic embeddings, FAISS vector search, evidence verification, grounded FLAN-T5 generation and source attribution in a single Streamlit application.

The project is intentionally transparent about its scope. It does not claim to answer every financial question or replace financial analysis. Instead, it provides a practical foundation for evidence-grounded document question answering, with clear paths for stronger retrieval, evaluation and production security in future versions.

16. References

1. Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS.
2. Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP.
3. Johnson, J., Douze, M. and Jégou, H. (2017). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data.
4. Raffel, C. et al. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. JMLR.
5. Microsoft Corporation. Annual Reports for fiscal years 2023, 2024 and 2025.
6. Streamlit documentation. https://docs.streamlit.io/
7. Hugging Face Transformers documentation. https://huggingface.co/docs/transformers/